Bölüm 7: Sözlükler ve Kümeler

SE 422 · Introduction to Data Science · Konu özeti

Kaynak slayt destesi: week4_Dictionaries_and_Sets
7 Kasım 2026

İçindekiler

Bölüm 7 · Sözlükler ve Kümeler 3

Bölümün amacı 3

Konu anlatımı 3

Sözlük nedir? (Slayt 1–3) 3

Değer almak, eklemek, silmek (Slayt 4–6) 3

len, karışık tipler, boş sözlük ve döngü (Slayt 7–8) 4

Sözlük metotları (Slayt 9–13) 4

Sözlük üreteci (Slayt 14–19) 5

Kümeler (Slayt 20–24) 5

Küme işlemleri (Slayt 25–30) 6

Küme üreteci (Slayt 31–34) 6

Nesneleri serileştirme: pickle (Slayt 35–39) 6

Sık yapılan hatalar 7

Kısa tekrar soruları 7

Cevaplar 8

Bölüm 7 · Sözlükler ve Kümeler

Bölümün amacı

Bu bölüm veriye **konumla değil anahtarla** erişmeyi (sözlük), **benzersiz öğe koleksiyonlarıyla** çalışmayı (küme) ve nesneleri dosyaya **serileştirmeyi** (pickle) öğretir. Sonunda şunları yapabilmelisin:

- Sözlük oluşturmak; değer almak, eklemek, değiştirmek, silmek; `get`, `items`, `keys`, `values`, `pop`, `popitem`, `clear` kullanmak
- Sözlük ve küme üreteçleri yazmak
- Küme oluşturmak; birleşim, kesişim, fark, simetrik fark, alt/üst küme işlemlerini yapmak
- `pickle` ile nesneleri dosyaya kaydetmek ve geri yüklemek

Konu anlatımı

Sözlük nedir? (Slayt 1–3)

Sözlük, her elemanı bir **anahtar** ve bir **değerden** oluşan koleksiyondur; anahtardan değere bir **eşlemedir**. Anahtar değiştirilemez (immutable) bir nesne olmalıdır (string, sayı, tuple); değerler herhangi bir tipte olabilir. `{anahtar: deger, ...}` ile yazılır.

PYTHON
```
phonebook = {'Chris': '555-1111', 'Katie': '555-2222', 'Joanne': '555-3333'}
print(phonebook)
print(phonebook['Katie'])
```

ÇIKTI
```
{'Chris': '555-1111', 'Katie': '555-2222', 'Joanne': '555-3333'}
555-2222
```

Değer almak, eklemek, silmek (Slayt 4–6)

- `d[anahtar]` değeri getirir; anahtar yoksa **KeyError**. `in` / `not in` önce kontrol etmeyi sağlar.
- `d[anahtar] = deger` yeni çift ekler; anahtar varsa değerini **değiştirir**.
- `del d[anahtar]` çifti siler; anahtar yoksa KeyError.

PYTHON

```python
phonebook = {'Chris': '555-1111'}
phonebook['Joe'] = '555-0123'
phonebook['Chris'] = '555-4444'
print('Joe' in phonebook, phonebook)
del phonebook['Joe']
print(phonebook)
```

ÇIKTI

```
True {'Chris': '555-4444', 'Joe': '555-0123'}
{'Chris': '555-4444'}
```

len, karışık tipler, boş sözlük ve döngü (Slayt 7–8)

`len` çift sayısını verir. Boş sözlük `{}` veya `dict()` ile oluşturulur. `for anahtar in d:` **anahtarlar** üzerinde dolaşır.

PYTHON

```python
test_scores = {'Kayla': [88, 92, 100], 'Luis': [95, 74, 81]}
print(len(test_scores))
for name in test_scores:
    print(name, sum(test_scores[name]) / len(test_scores[name]))
bos = {}
bos[42] = 'cevap'
print(bos)
```

ÇIKTI

```
2
Kayla 93.33333333333333
Luis 83.33333333333333
{42: 'cevap'}
```

Sözlük metotları (Slayt 9–13)

Metot	Ne yapar?
`get(k, varsayilan)`	değeri getirir; anahtar yoksa varsayılanı döndürür, **KeyError fırlatmaz**
`items()`	(anahtar, değer) tuple'ları
`keys()` / `values()`	anahtarlar / değerler
`pop(k, varsayilan)`	değeri döndürür ve çifti siler
`popitem()`	**son eklenen** çifti döndürür ve siler
`clear()`	sözlüğü boşaltır

PYTHON
```python
phonebook = {'Chris': '555-1111', 'Katie': '555-2222'}
print(phonebook.get('Andy', 'Bulunamadı'))
for key, value in phonebook.items():
    print(key, value)
d = {'a': 1, 'b': 2, 'c': 3}
print(list(d.keys()), d.pop('a'), d.popitem(), list(d.values()))
```

ÇIKTI
```
Bulunamadı
Chris 555-1111
Katie 555-2222
['a', 'b', 'c'] 1 ('c', 3) [2]
```

Sözlük üreteci (Slayt 14–19)

`{anahtar: deger for oge in dizi}` bir diziden sözlük üretir; `if` ile seçim yapılabilir. `{k: v for k, v in d.items()}` sözlüğü kopyalar.

PYTHON
```python
numbers = [1, 2, 3, 4]
print({item: item ** 2 for item in numbers})
print({item: len(item) for item in ['Jeremy', 'Kate', 'Peg']})
pop = {'New York': 8398748, 'Phoenix': 1660272, 'Houston': 2325502}
print({k: v for k, v in pop.items() if v > 2000000})
```

ÇIKTI
```
{1: 1, 2: 4, 3: 9, 4: 16}
{'Jeremy': 6, 'Kate': 4, 'Peg': 3}
{'New York': 8398748, 'Houston': 2325502}
```

Kümeler (Slayt 20–24)

Küme, matematiksel küme gibi davranır: öğeler **benzersizdir**, küme **sırasızdır** (indekslenemez). `set()` boş küme, `set(liste)` listeden küme oluşturur; string verilirse her karakter bir öğe olur. `add` bir öğe, `update` birden çok öğe ekler. `remove` öğe yoksa **KeyError** fırlatır, `discard` fırlatmaz.

PYTHON
```python
s = set([1, 2, 2, 3, 3, 3])
print(s, len(s), sorted(set('aabbc')))
s.add(4)
s.update([5, 6])
s.discard(10)
s.remove(2)
print(s, 3 in s)
```

ÇIKTI

```
{1, 2, 3} 3 ['a', 'b', 'c']
{1, 3, 4, 5, 6} True
```

Küme işlemleri (Slayt 25–30)

İşlem	Metot	Operatör	
Birleşim	`union`	`	`	`
Kesişim	`intersection`	`&`	
Fark	`difference`	`-`	
Simetrik fark	`symmetric_difference`	`^`	
Alt küme / üst küme	`issubset` / `issuperset`	`<=` / `>=`	

PYTHON

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(sorted(a | b), sorted(a & b), sorted(a - b), sorted(a ^ b))
print({1, 2} <= a, a.issuperset({1, 2}), a.union(b) == a | b)
```

ÇIKTI

```
[1, 2, 3, 4, 5, 6] [3, 4] [1, 2] [1, 2, 5, 6]
True True True
```

Küme üreteci (Slayt 31–34)

Liste üreteci gibi yazılır ama süslü parantez kullanılır; sonuç bir kümedir (tekrarlar kendiliğinden düşer).

PYTHON

```
set1 = set([1, 20, 2, 40, 3, 50])
print(sorted({item for item in set1 if item < 10}))
print(sorted({len(w) for w in ['ab', 'cd', 'efg']}))
```

ÇIKTI

```
[1, 2, 3]
[2, 3]
```

Nesneleri serileştirme: pickle (Slayt 35–39)

Serileştirme (pickling), bir nesneyi dosyada saklanabilecek bayt akışına dönüştürmektir. Adımlar: `import pickle` → dosyayı **ikili yazma** (`'wb'`) için aç → `pickle.dump(nesne, dosya)` → kapat. Geri almak (**unpickling**) için `'rb'` ile açıp `pickle.load(dosya)` çağrılır. Aynı dosyaya birden çok nesne yazılıp sırayla okunabilir.

PYTHON
```python
import pickle
phonebook = {'Chris': '555-1111', 'Katie': '555-2222'}
with open('phonebook.dat', 'wb') as f:
    pickle.dump(phonebook, f)
    pickle.dump([1, 2, 3], f)
with open('phonebook.dat', 'rb') as f:
    print(pickle.load(f))
    print(pickle.load(f))
```

ÇIKTI
```
{'Chris': '555-1111', 'Katie': '555-2222'}
[1, 2, 3]
```

Sık yapılan hatalar

- **Olmayan anahtarı `[]` ile istemek** → `KeyError`; `get` ya da `in` kullan.

PYTHON
```python
d = {'elma': 3}
print(d['kiraz'])
```

ÇIKTI
```
KeyError: 'kiraz'
```

- **Değiştirilebilir nesneyi anahtar yapmak:** liste anahtar olamaz → `TypeError: unhashable type`.
- **Kümeyi indekslemek:** `s[0]` → `TypeError`; kümeler sırasızdır.
- **`{}`'nin boş küme olduğunu sanmak:** `{}` boş **sözlüktür**; boş küme `set()` ile oluşturulur.

PYTHON
```python
print(type({}), type(set()))
```

ÇIKTI
```
<class 'dict'> <class 'set'>
```

- **pickle dosyasını metin modunda açmak:** `'wb'` / `'rb'` gerekir.

Kısa tekrar soruları

1 `d = {'a': 1}` için `d.get('b', 0)` ve `d['b']` ne verir?

2 Bir sözlükte anahtar olarak hangi tipler kullanılabilir?

3 `{x: x % 3 for x in range(5)}` nedir?

4 `set('mississippi')` kaç öğe içerir?

5 `{1, 2, 3} ^ {2, 3, 4}` nedir?

6 `remove` ile `discard` arasındaki fark nedir?

Cevaplar

1 `0` ve `KeyError`.

2 Değiştirilemez (immutable) tipler: string, sayı, tuple vb.

3 `{0: 0, 1: 1, 2: 2, 3: 0, 4: 1}`

4 4 öğe: `m`, `i`, `s`, `p`.

5 `{1, 4}`

6 Öğe kümede yoksa `remove` KeyError fırlatır, `discard` sessizce hiçbir şey yapmaz.